

# **ALGORITMOS e LÓGICA DE PROGRAMAÇÃO (AeLP)**

## **AULA 04**

# ESTRUTURAS DE DADOS

- Vetores
  - » O vetor é um conjunto de variáveis do mesmo tipo referenciadas por um único nome seguido de um índice.
  - $\text{notas}[2] \leftarrow 3$

The diagram illustrates an array structure with indices and values. The array is represented as a table with two columns. The first column contains indices, and the second column contains values. The indices are grouped into four sets, each labeled with a variable name and an index in blue text:  $\text{notas}[n]$ ,  $\text{notas}[2]$ ,  $\text{notas}[1]$ , and  $\text{notas}[0]$ . The values are either empty or a sequence of digits.

|                   |         |          |
|-------------------|---------|----------|
| $\text{notas}[n]$ | 2293505 |          |
|                   | 2293504 |          |
|                   | 2293503 |          |
|                   | 2293502 |          |
| $\text{notas}[2]$ | 2293501 | 00000000 |
|                   | 2293500 | 00000000 |
|                   | 2293599 | 00000000 |
|                   | 2293598 | 00000111 |
| $\text{notas}[1]$ | 2293597 |          |
|                   | 2293596 |          |
|                   | 2293595 |          |
|                   | 2293594 |          |
| $\text{notas}[0]$ | 2293593 |          |
|                   | 2293592 |          |
|                   | 2293591 |          |
|                   | 2293590 |          |

# ESTRUTURAS DE DADOS

- Vetores

- » Declaração:

- `<var> : vetor[<li>..<lf>] de <tipoPrimitivo>`

- » Em que:

- `<identificador>` é o nome da variável do tipo vetor
- `<li>` é o limite inicial do vetor
- `<lf>` é o limite final do vetor
- `<tipoPrimitivo>` é o tipo de dado básico que será armazenado

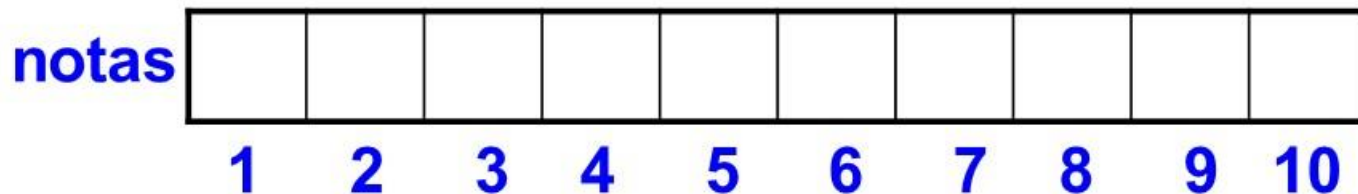
# ESTRUTURAS DE DADOS

- Vetores

- » Declaração:

- notas : vetor[1..10] de inteiro

- » Esta declaração aloca memória para armazenar 10 inteiros e informa que “notas” é o nome para acessar os 10 elementos

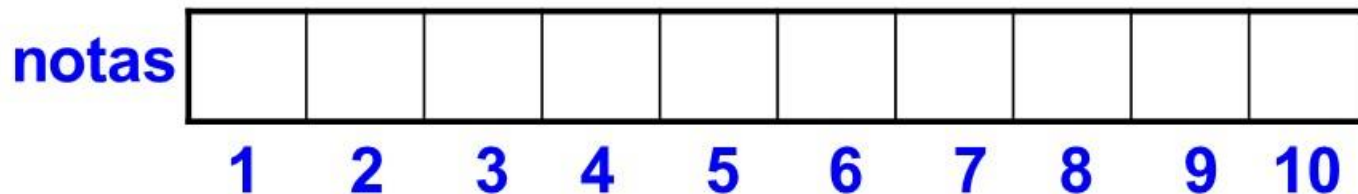


# ESTRUTURAS DE DADOS

- Vetores

- » Manipulação:

- Cada posição do vetor pode ser acessada a partir de um índice, que indica essa posição.
- As posições do vetor funcionam como uma variável comum, que armazena um valor único.



# ESTRUTURAS DE DADOS

- Vetores

- » Manipulação:

- notas[2] ← 8
- num ← 9 //num : inteiro
- notas[3] ← num
- leia (notas[6]) //usuário digitou 5
- escreva(notas[6] + 2) //mostra o valor 7

|       |   |   |   |   |   |   |   |   |   |    |
|-------|---|---|---|---|---|---|---|---|---|----|
| notas |   | 8 | 9 |   |   | 5 |   |   |   |    |
|       | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |

# ESTRUTURAS DE DADOS

algoritmo "semnome"

// Função :

// Autor :

// Data : 29/09/2013

// Seção de Declarações

var

n1,n2,n3,n4,n5,n6,n7,n8,n9,n10: inteiro

soma,media :real

Inicio

soma <-0

escreva ("valor 1: ")

leia(n1)

escreva ("valor 2: ")

leia(n2)

escreva ("valor 3: ")

leia(n3)

escreva ("valor 4: ")

leia(n4)

escreva ("valor 5: ")

leia(n5)

escreva ("valor 6: ")

leia(n6)

escreva ("valor 7: ")

leia(n7)

escreva ("valor 8: ")

leia(n8)

escreva ("valor 9: ")

leia(n9)

escreva ("valor 10: ")

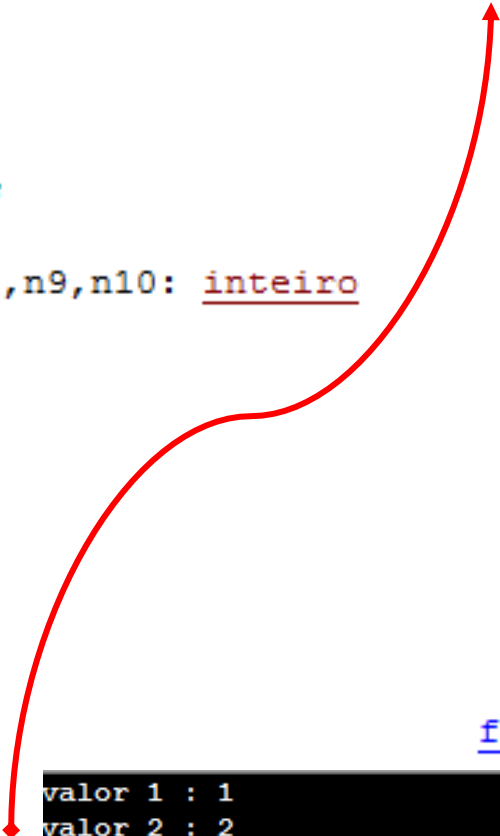
leia(n10)

soma <- n1+n2+n3+n4+n5+n6+n7+n8+n9+n10

media <- soma/10

escreval(media)

fimalgoritmo



```
valor 1 : 1
valor 2 : 2
valor 3 : 3
valor 4 : 4
valor 5 : 5
valor 6 : 6
valor 7 : 7
valor 8 : 8
valor 9 : 9
valor 10 : 10
5.5
```

\*\*\* Fim da execução.

\*\*\* Feche esta janela para retornar ao Visualg.

# ESTRUTURAS DE DADOS

algoritmo "semnome"

// Função :

// Autor :

// Data : 29/09/2013

// Seção de Declarações

var

vet: vetor[1..10] de inteiro

i :inteiro

soma,media :real

Inicio

soma<- 0

para i de 1 ate 10 passo 1 faca

    escreva ("valor", i , " : ")

    leia(vet[i])

    soma <- soma + vet[i]

fimpara

media <- soma/10

escreval(media)

fimalgoritmo

```
valor 1 : 1
valor 2 : 2
valor 3 : 3
valor 4 : 4
valor 5 : 5
valor 6 : 6
valor 7 : 7
valor 8 : 8
valor 9 : 9
valor 10 : 10
5.5
```

\*\*\* Fim da execução.

\*\*\* Feche esta janela para retornar ao Visualg.



# ESTRUTURAS DE DADOS

- Exercício - aula9\_9
  - » Elabore um algoritmo que execute as seguintes ações:
    - 1) Receba 5 valores e armazene em vet1
    - 2) Receba 5 valores e armazene em vet2
    - 3) Apresente a soma de cada posição dos vetores vet1 e vet2

```
vet1: pos 1 : 1  
vet1: pos 2 : 2  
vet1: pos 3 : 3  
vet1: pos 4 : 4  
vet1: pos 5 : 5
```

```
vet2: pos 1 : 6  
vet2: pos 2 : 7  
vet2: pos 3 : 8  
vet2: pos 4 : 9  
vet2: pos 5 : 10
```

```
soma:  
7 9 11 13 15
```

# ESTRUTURAS DE DADOS

```
algoritmo "semnome"  
// Função :  
// Autor :  
// Data : 29/09/2013  
// Seção de Declarações  
var  
vet1,vet2: vetor[1..5] de inteiro  
i :inteiro  
inicio  
para i de 1 ate 5 passo 1 faca  
    escreva ("vet1: pos", i , " : ")  
    leia(vet1[i])  
fimpara  
escreval()  
para i de 1 ate 5 passo 1 faca  
    escreva ("vet2: pos", i , " : ")  
    leia(vet2[i])  
fimpara  
escreval()  
escreval("soma:")  
para i de 1 ate 5 passo 1 faca  
    escreva (vet1[i]+vet2[i])  
fimpara  
fimalgoritmo
```

```
vet1: pos 1 : 1  
vet1: pos 2 : 2  
vet1: pos 3 : 3  
vet1: pos 4 : 4  
vet1: pos 5 : 5  
  
vet2: pos 1 : 6  
vet2: pos 2 : 7  
vet2: pos 3 : 8  
vet2: pos 4 : 9  
vet2: pos 5 : 10  
  
soma:  
  7 9 11 13 15  
*** Fim da execução.  
*** Feche esta janela para retornar ao Visualg.
```

# ESTRUTURAS DE DADOS

- Exercício - aula9\_11

» Elabore um algoritmo que receba 5 valores, armazene em um vetor e imprima na tela:

- O vetor em ordem inversa
- O quadrado de cada valor
- Os valores maiores que 3
- A posição dos valores pares.

Valor 1 : 2

Valor 2 : 6

Valor 3 : 1

Valor 4 : 8

Valor 5 : 3

Inverso:

3 8 1 6 2

Quadrado:

4 36 1 64 9

Maiores que 3:

6 8

Índice dos pares:

1 2 4

# ESTRUTURAS DE DADOS

```
algoritmo "semnome"
var
vet: vetor[1..5] de inteiro
i :inteiro
inicio
para i de 1 ate 5 passo 1 faca
    escreva("Valor",i," : ")
    leia(vet[i])
fimpara
escreval("Inverso: ")
para i de 5 ate 1 passo -1 faca
    escreva (vet[i])
fimpara
escreval()
    escreval("Quadrado:")
para i de 1 ate 5 passo 1 faca
    escreva (vet[i]*vet[i])
fimpara
```

```
escreval()
escreval("Maiores que 3:")
para i de 1 ate 5 passo 1 faca
    se(vet[i]>3) entao
        escreva (vet[i])
    fimse
fimpara
escreval()
escreval("Índice dos pares:")
para i de 1 ate 5 passo 1 faca
    se(vet[i] mod 2 = 0) entao
        escreva (i)
    fimse
fimpara
fimalgoritmo
```



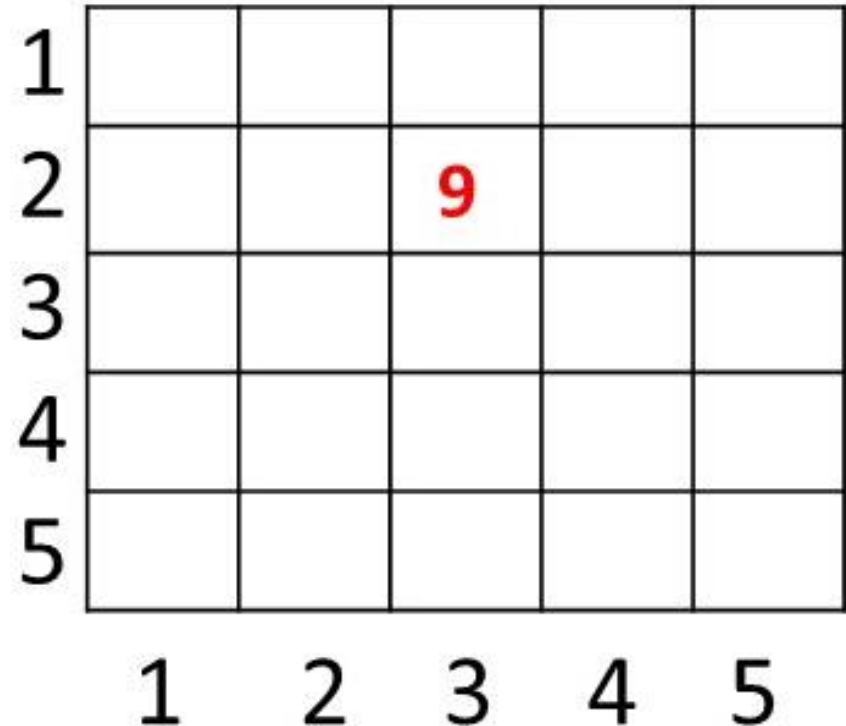
```
Valor 1 : 1
Valor 2 : 2
Valor 3 : 3
Valor 4 : 4
Valor 5 : 5
Inverso:
 5 4 3 2 1
Quadrado:
 1 4 9 16 25
Maiores que 3:
 4 5
Índice dos pares:
 2 4
*** Fim da execução.
*** Feche esta janela para retornar ao Visualg.
```

# ESTRUTURAS DE DADOS

- Matrizes

- » A matriz é um conjunto de variáveis multidimensionais do mesmo tipo referenciadas por um único nome seguido de dois ou mais índices.

- `mat[2][3] ← 9`



|   |   |   |   |   |   |
|---|---|---|---|---|---|
| 1 |   |   |   |   |   |
| 2 |   |   | 9 |   |   |
| 3 |   |   |   |   |   |
| 4 |   |   |   |   |   |
| 5 |   |   |   |   |   |
|   | 1 | 2 | 3 | 4 | 5 |

# ESTRUTURAS DE DADOS

- Matrizes

- » Declaração:

- `<var> : vetor[<li>..<lf>, <li..lf>] de <tipoPrimitivo>`

- » Em que:

- `<identificador>` é o nome da variável do tipo vetor
- `<li>` é o limite inicial do vetor
- `<lf>` é o limite final do vetor
- `<tipoPrimitivo>` é o tipo de dado básico que será armazenado

# ESTRUTURAS DE DADOS

- Matrizes

- » Declaração:

- `mat : vetor[1..3,1..10]` de inteiro

- » Esta declaração aloca memória para armazenar 30 inteiros e informa que “mat” é o nome para acessar os elementos dispostos em linha e coluna.

A diagram of a 3x10 matrix. The grid consists of 3 rows and 10 columns. To the left of the grid, the row indices 1, 2, and 3 are listed vertically. Below the grid, the column indices 1 through 10 are listed horizontally. All indices are in blue text.

|   |   |   |   |   |   |   |   |   |   |    |
|---|---|---|---|---|---|---|---|---|---|----|
| 1 |   |   |   |   |   |   |   |   |   |    |
| 2 |   |   |   |   |   |   |   |   |   |    |
| 3 |   |   |   |   |   |   |   |   |   |    |
|   | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |

# ESTRUTURAS DE DADOS

- Matrizes

» Manipulação:

- `mat[1,8] ← 55`
- `num ← 15` //num : inteiro
- `mat[3,4] ← num`
- `leia (mat[2,10])` //usuário digitou 31
- `escreva(mat[2,10] + 5)` //mostra o valor 36

|   |   |   |   |    |   |   |   |    |   |    |
|---|---|---|---|----|---|---|---|----|---|----|
| 1 |   |   |   |    |   |   |   | 55 |   |    |
| 2 |   |   |   |    |   |   |   |    |   | 31 |
| 3 |   |   |   | 15 |   |   |   |    |   |    |
|   | 1 | 2 | 3 | 4  | 5 | 6 | 7 | 8  | 9 | 10 |



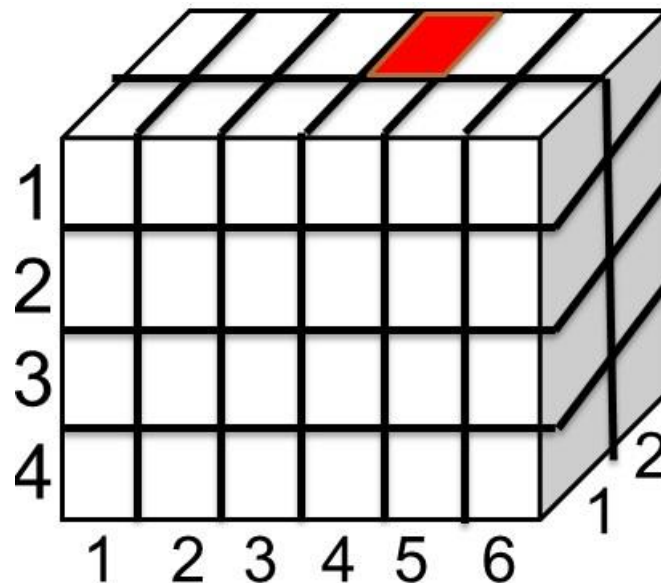
# ESTRUTURAS DE DADOS

- Matrizes

» Matriz de 3 dimensões:

- `mat : vetor[1..4,1..6, 1..2]` de inteiro
- Esta declaração aloca memória para armazenar uma matriz de 48 elementos ( $4 * 6 * 2$ ) que podem ser acessados pelo nome `mat` seguido de 3 índices.

- Ex: `mat[1][4][2]`



# ESTRUTURAS DE DADOS

- Exercício
  - » Preencher uma matriz 3x4 com valores fornecidos pelo usuário e imprimi-la formatada

Elemento 1, 1: 1

Elemento 1, 2: 2

Elemento 1, 3: 3

Elemento 2, 1: 4

Elemento 2, 2: 5

Elemento 2, 3: 6

Elemento 3, 1: 7

Elemento 3, 2: 8

Elemento 3, 3: 9

1 2 3

4 5 6

7 8 9

# ESTRUTURAS DE DADOS

```
algoritmo "semnome"  
// Função :  
// Autor :  
// Data : 18/10/2013  
// Seção de Declarações  
var  
mat:vetor[1..3,1..3] de inteiro  
i,j :inteiro  
inicio  
para i de 1 ate 3 passo 1 faca  
    para j de 1 ate 3 passo 1 faca  
        escreva("Elemento",i,"",j," : ")  
        leia(mat[i,j])  
    fimpara  
fimpara  
para i de 1 ate 3 passo 1 faca  
    para j de 1 ate 3 passo 1 faca  
        escreva(mat[i,j])  
    fimpara  
escreval          //formata a matriz  
fimpara  
fimalgoritmo
```

```
Elemento 1, 1: 1  
Elemento 1, 2: 2  
Elemento 1, 3: 3  
Elemento 2, 1: 4  
Elemento 2, 2: 5  
Elemento 2, 3: 6  
Elemento 3, 1: 7  
Elemento 3, 2: 8  
Elemento 3, 3: 9
```

```
1 2 3  
4 5 6  
7 8 9
```

\*\*\* Fim da execução.

\*\*\* Feche esta janela para retornar ao Visualg.

# ESTRUTURAS DE DADOS

- Exercício
  - » Preencher uma matriz 5x5 com valores aleatórios
  - » Imprimir “Para sair digite 0,0”
  - » Laço:
    - Imprimi-la formatada, alinhando as colunas
    - Solicitar uma posição ao usuário e apresentar o conteúdo daquela posição.
    - Sair, caso seja digitado 0,0

Para sair digite 0,0

59 78 83 99 42

88 96 17 12 77

80 24 34 8 68

4 67 87 79 26

3 69 28 90 53

Digite: linha, coluna

3

5

O valor é: 68

59 78 83 99 42

88 96 17 12 77

80 24 34 8 68

4 67 87 79 26

3 69 28 90 53

Digite: linha, coluna

0

0

# ESTRUTURAS DE DADOS

algoritmo "semnome"

// Função :

// Autor :

// Data : 18/10/2013

// Seção de Declarações

var

mat:vetor[1..5,1..5] de inteiro

i,j,lin,col : inteiro

Sair : logico

inicio

para i de 1 ate 5 passo 1 faca

para j de 1 ate 5 passo 1 faca

mat[i,j]<-int(rand\*100)

fimpara

fimpara

escreval("Para sair digite 0,0")

Para sair digite 0,0

22 92 99 37 41

9 30 49 23 46

9 72 89 56 17

71 16 61 17 8

45 35 98 75 34

Digite: linha e coluna

1

1

O valor é: 22

22 92 99 37 41

9 30 49 23 46

9 72 89 56 17

71 16 61 17 8

45 35 98 75 34

Digite: linha e coluna

0

0

\*\*\* Fim da execução.

\*\*\* Feche esta janela para retornar ao Visualg.

repita

para i de 1 ate 5 passo 1 faca

para j de 1 ate 5 passo 1 faca

se(mat[i,j]>9)entao

escreva(mat[i,j])

senao

escreva(" ",mat[i,j])

fimse

fimpara

escreval

fimpara

sair<-falso

escreval("Digite: linha e coluna")

leia(lin,col)

se((lin=0) e (col=0)) entao

sair<-verdadeiro

senao

escreval("O valor é:", mat[lin,col])

escreval

fimse

ate(sair)

fimalgoritmo

# ESTRUTURAS DE DADOS

- Exercício
  - » Elabore um algoritmo para venda de cadeiras em um teatro
  - » Apresente uma matriz 9x9 com “.” para indicar as cadeira livres e “X” para as cadeiras reservadas.
  - » Laço:
    - Imprimi-la formatada, apresentando o número das linhas e das colunas
    - Solicitar uma posição ao usuário, marcar com “X” e exibir “Cadeira reservada” ou “Cadeira indisponível”
    - Sair, caso seja digitado um local inexistente

**IDÊNTICO AO EXEMPLO <<vire>>**

# ESTRUTURAS DE DADOS

Para sair digite um local inexistente

1 2 3 4 5 6 7 8 9

1 . . . . .

2 . . . . .

3 . . . . .

4 . . . . .

5 . . . . .

6 . . . . .

7 . . . . .

8 . . . . .

9 . . . . .

Posição da cadeira: linha, coluna

2  
6

Cadeira reservada...

1 2 3 4 5 6 7 8 9

1 . . . . .

2 . . . . X . .

3 . . . . .

4 . . . . .

5 . . . . .

6 . . . . .

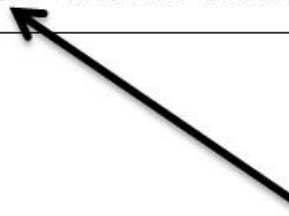
7 . . . . .

8 . . . . .

9 . . . . .

Posição da cadeira: linha, coluna

1  
10

 Ou, Cadeira indisponível

# ESTRUTURAS DE DADOS

```
algoritmo "semnome"
// Função :
// Autor :
// Data : 18/10/2013
// Seção de Declarações
var
mat:vetor[1..9,1..9] de caracter
i,j,lin,col:inteiro
sair:logico
inicio
para i de 1 ate 9 passo 1 faca
    para j de 1 ate 9 passo 1 faca
        mat[i,j]<-"."
    fimpara
fimpara
escreval("Para sair digite um local inexistente")
repita
    escreval("    1 2 3 4 5 6 7 8 9")
    para i de 1 ate 9 passo 1 faca
        escreva(i," ")
        para j de 1 ate 9 passo 1 faca
            escreva(mat[i,j]," ")
        fimpara
    fimpara
escreval
fimpara
```

```
sair<-falso
escreval("Posição da cadeira: linha e coluna")
leia(lin,col)
se((lin<1)ou(lin>9)ou(col<1)ou(col>9)) entao
    sair<-verdadeiro
senao
    se(mat[lin,col]=".") entao
        mat[lin,col]<- "X"
        escreval("Cadeira reservada...")
        escreval
    senao
        escreval("Cadeira indisponível...")
        escreval
    fimse
fimse
ate(sair)
fimalgoritmo
```

```
Para sair digite um local inexistente
 1 2 3 4 5 6 7 8 9
1 . . . . .
2 . . . . .
3 . . . . .
4 . . . . .
5 . . . . .
6 . . . . .
7 . . . . .
8 . . . . .
9 . . . . .
Posição da cadeira: linha e coluna
0
0
```

```
*** Fim da execução.
*** Feche esta janela para retornar ao Visualg.
```